

Reporte de laboratorio S5

Mexi Banco v05.1 con Docker Compose

Alumna: Alejandra Melissa Saucedo Serralde

Repositorio del proyecto: github.com/OscarRuiz21/mexi-banco

Paso 1 · Clonar el repositorio

```
git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
cd mexi-banco
```

- Lo cloné en mi carpeta de usuario, fuera de sd-2027-1, para no mezclarlo con mis entregas.
- Git avisó que estaba en "detached HEAD". No era un error: v05.1 es una versión etiquetada, no una rama.
- **Problema que tuve.** En el primer docker compose up --build, la construcción falló en la línea RUN ./mvnw dependency:go-offline -B con el mensaje "/bin/sh: ./mvnw: not found", aunque el archivo mvnw sí estaba en la carpeta. Mi explicación es que Git en Windows convirtió los saltos de línea de LF a CRLF y el script no se pudo ejecutar dentro del contenedor Linux. Borré la carpeta y la cloné otra vez con el comando de abajo; con eso la construcción terminó bien, lo que respalda esa explicación.

```
git clone -c core.autocrlf=false --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
```

Paso 2 · Levantarlo con Compose

```
docker compose up --build -d
docker compose ps
```

- El up creó la imagen mexi-banco-app, la red mexibanco, el volumen mexi-banco_mexibanco_datos y los dos contenedores.
- db llegó a Healthy antes de que app arrancara (Started). Más adelante, en el docker-compose.yml, vi que app depende de db con condition: service_healthy.
- docker compose ps mostró db y app en (healthy), no solo "running".

- La primera construcción tardó unos 79 segundos (descarga de imágenes base y dependencias de Maven). Después de un down y un up, la imagen se construyó en 6.6 segundos. Lo más probable es que Docker reutilizó capas en caché; no lo verifiqué más a fondo.

```

UCRT64/c/Users/Melis/mexi-banco
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco ((v05.1))
$ docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1      Removed      0.0s
✓ Container mexi-banco-db-1       Removed      0.1s
✓ Volume mexi-banco_mexibanco_datos Removed      0.0s
✓ Network mexibanco               Removed      0.2s

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco ((v05.1))
$ docker compose up -d
[+] up 4/4
✓ Network mexibanco               Created      0.0s
✓ Volume mexi-banco_mexibanco_datos Created      0.0s
✓ Container mexi-banco-db-1       Healthy      6.1s
✓ Container mexi-banco-app-1       Started      6.3s

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco ((v05.1))
$ docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE  CREATED          STATUS
mexi-banco-app-1    mexi-banco-app                     "java -jar app.jar"                  app       30 seconds ago  Up 23 seco
nds (healthy)      0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1    postgres:17-alpine                 "docker-entrypoint.s..."            db        30 seconds ago  Up 29 seco
nds (healthy)      5432/tcp

```

down -v, up -d y docker compose ps, con db y app en healthy.

Paso 3 · Probar los endpoints en Postman

Importé la colección mexi-banco-v05.1.postman_collection.json y mandé las peticiones siguiendo el orden numérico de las carpetas (Postman las muestra desordenadas en la lista). Esto respondió cada una:

Petición	Código	Respuesta
1 · Salud de la app	200	status: UP
1 · Abrir cuenta origen	201	Cuenta de Ana, id 1, saldo 1000
1 · Consultar cuenta origen	200	La misma cuenta, saldo 1000.00
2 · Abrir cuenta destino	201	Cuenta de Beto, id 2, saldo 500

Petición	Código	Respuesta
2 · Transferir 200	201	Transferencia id 1, de la CLABE de Ana a la de Beto, monto 200
2 · Consultar origen	200	Ana: 800.00
2 · Consultar destino	200	Beto: 700.00
2 · Consultar la transferencia	200	Transferencia id 1, monto 200.00
3 · SPEI de 50 (primer envío)	201	id 1, estado ENVIADO, banco BBVA
3 · SPEI de 50 (reintento)	201	Mismo id 1 y misma llave
3 · Consultar origen	200	Ana: 750.00
4 · Estado de cuenta del origen	200	Tres movimientos (ver observaciones)
4 · Avisos del origen	200	Dos avisos (ver observaciones)
5 · CLABE que no existe	404	Problem Details: "No existe la CLABE..."
5 · CLABE repetida	409	Problem Details: "Ya existe una cuenta con la CLABE..."
5 · Cuerpo sin clabe	400	Formato de Spring (timestamp, error, path)
5 · Transferir a la misma cuenta	422	Problem Details: "La cuenta origen y la destino son la misma"
5 · Los errores no movieron dinero	200	Ana sigue en 750.00
5 · Saldo insuficiente	422	Problem Details: "Saldo insuficiente en la CLABE..."
5 · SPEI sin Idempotency-Key	400	Formato de Spring (timestamp, error, path)

Capturas · carpeta 1 (Arriba y respondiendo)

The screenshot shows a REST client interface with a sidebar on the left containing a collection named 'Mexi Banco v05.1'. The selected item is 'GET Salud de la app'. The main panel displays the response for this request, which is a 200 status code. The response body is in JSON format and contains the following data:

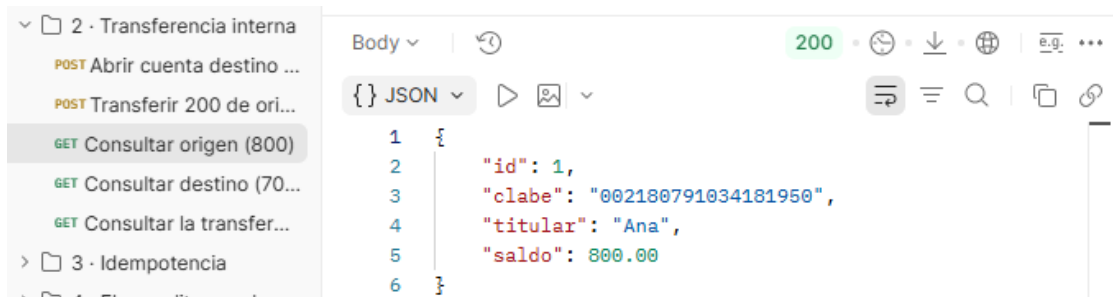
```

{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}

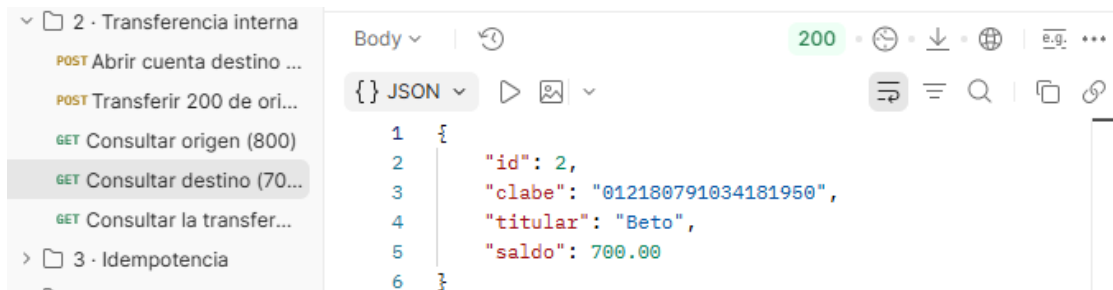
```

Salud de la app — 200, status UP.

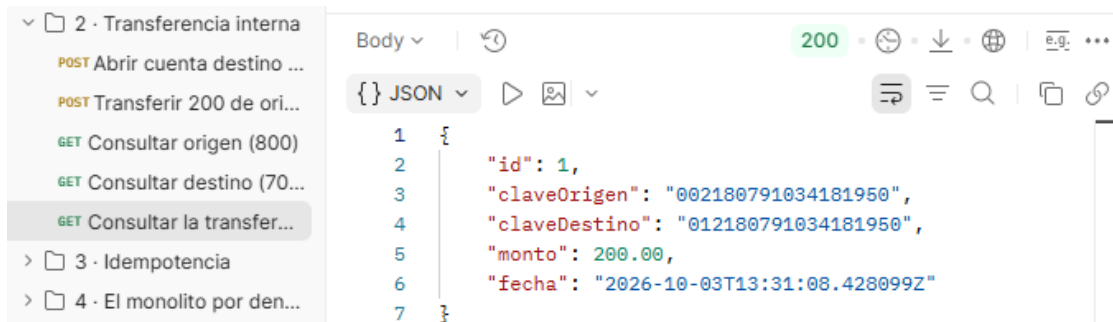




Consultar origen — 200, Ana: 800.00.

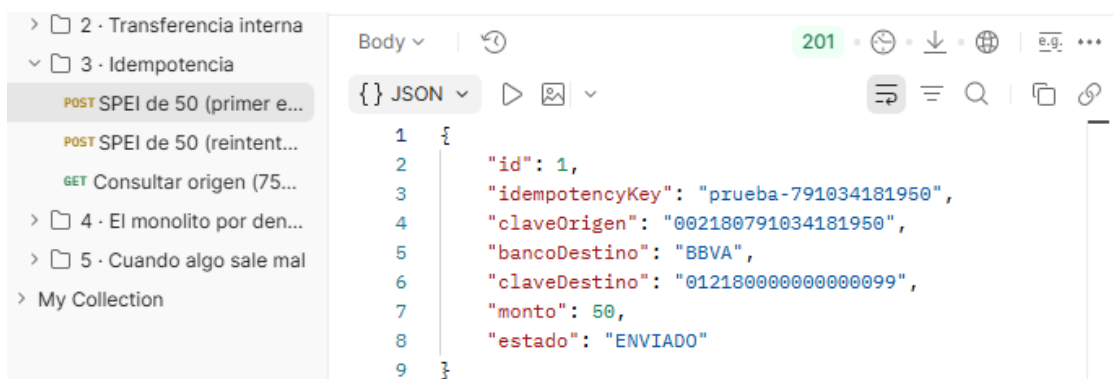


Consultar destino — 200, Beto: 700.00.

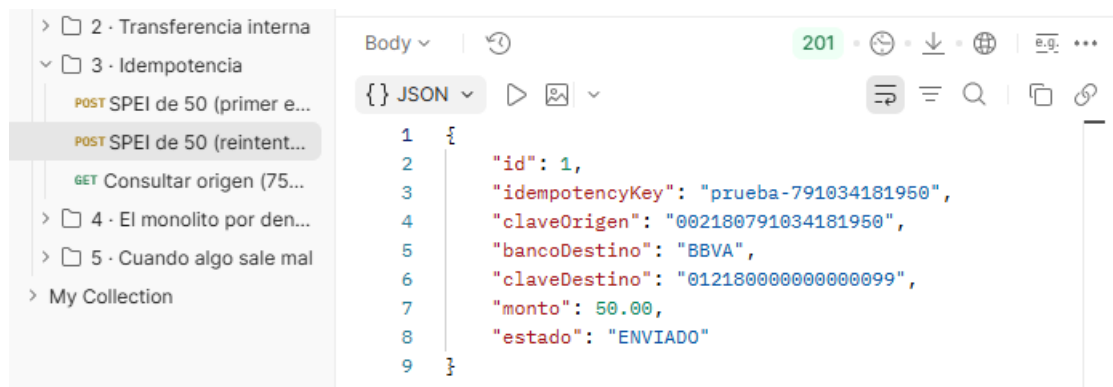


Consultar la transferencia — 200, misma transferencia id 1.

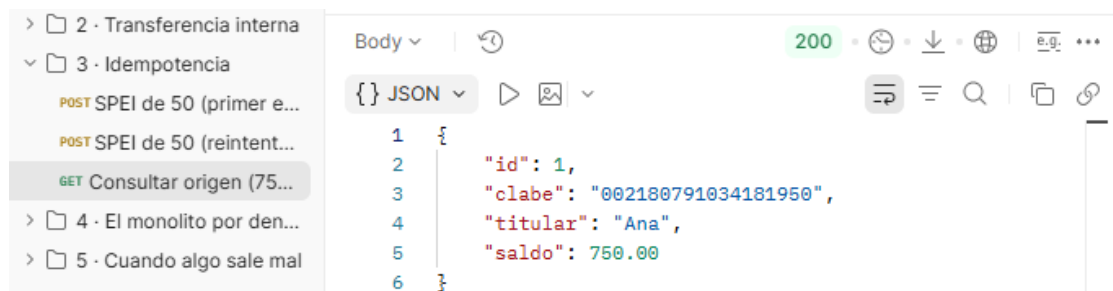
Capturas · carpeta 3 (Idempotencia)



SPEI de 50 (primer envío) — 201, id 1, llave idempotencyKey.



SPEI de 50 (reintento) — 201, mismo id 1 y misma llave.



Consultar origen — 200, Ana: 750.00 (se cobró una sola vez).

Capturas · carpeta 4 (El monolito por dentro)

Left sidebar (Collection 4 - El monolito por dentro):

- GET Estado de cuenta del...
- GET Avisos del origen
- 5 - Cuando algo sale mal
- My Collection

Response (200):

```
{
  "id": 5,
  "clabeCuenta": "002180791034181950",
  "tipo": "SPEI_ENVIADO",
  "monto": -50.00,
  "saldoResultante": 750.00,
  "fecha": "2026-10-03T13:32:55.937281Z",
  "referencia": "BBVA - 012180000000000099"
},
{
  "id": 3,
  "clabeCuenta": "002180791034181950",
  "tipo": "TRANSFERENCIA_ENVIADA",
  "monto": -200.00,
  "saldoResultante": 800.00,
  "fecha": "2026-10-03T13:31:08.424668Z",
  "referencia": "a 012180791034181950"
},
{
  "id": 1,
  "clabeCuenta": "002180791034181950",
  "tipo": "DEPOSITO",
  "monto": 1000.00,
  "saldoResultante": 1000.00,
  "fecha": "2026-10-03T13:29:42.692646Z",
  "referencia": "saldo inicial"
}
```

Estado de cuenta del origen — 200, tres movimientos.

Left sidebar (Collection 4 - El monolito por dentro):

- GET Estado de cuenta del...
- GET Avisos del origen
- 5 - Cuando algo sale mal
- My Collection

Response (200):

```
{
  "id": 3,
  "clabeCuenta": "002180791034181950",
  "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
  "fecha": "2026-10-03T13:32:56.347731Z"
},
{
  "id": 1,
  "clabeCuenta": "002180791034181950",
  "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
  "fecha": "2026-10-03T13:31:08.434303Z"
}
```

Avisos del origen — 200, dos avisos.

Capturas · carpeta 5 (Cuando algo sale mal)

Body ▾ | ↺ 404 = 🕒 = ⬇ = 🌐 | e.g. ...

✦ Use a valid CLABE Diagnose Decline ... ⚙

{ } JSON ▾ ▶ 🕒 ▾

```
1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 999999999999999999",
6   "instance": "/cuentas/999999999999999999"
7 }
```

CLABE que no existe — 404, Problem Details.

Body ▾ | ↺ 409 = 🕒 = ⬇ = 🌐 | e.g. ...

✦ Resolve 409 Conflict error Diagnose Decline ... ⚙

{ } JSON ▾ ▶ 🕒 ▾

```
1 {
2   "type": "about:blank",
3   "title": "Conflict",
4   "status": 409,
5   "detail": "Ya existe una cuenta con la CLABE
              002180791034181950",
6   "instance": "/cuentas"
7 }
```

CLABE repetida — 409, Problem Details.

Body ▾ | ↺ 400 = 🕒 = ⬇ = 🌐 | e.g. ...

✦ Fix 400 Bad Request error Diagnose Decline ... ⚙

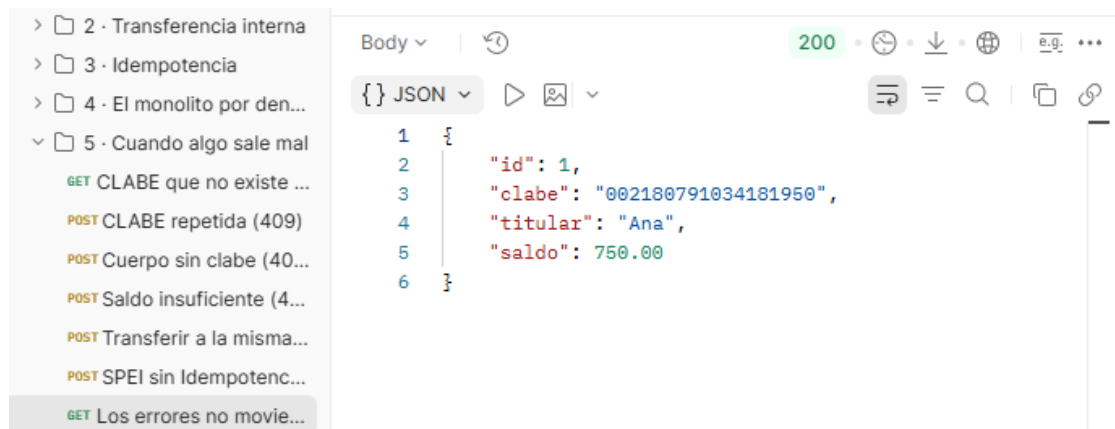
{ } JSON ▾ ▶ 🕒 ▾

```
1 {
2   "timestamp": "2026-10-03T13:35:38.876+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/cuentas"
6 }
```

Cuerpo sin clabe — 400, formato por defecto de Spring.



Transferir a la misma cuenta — 422, Problem Details.



Los errores no movieron dinero — 200, Ana sigue en 750.00.



Saldo insuficiente — 422, Problem Details.

```
1 {  
2   "timestamp": "2026-10-03T13:37:48.168+00:00",  
3   "status": 400,  
4   "error": "Bad Request",  
5   "path": "/spei"  
6 }
```

SPEI sin Idempotency-Key — 400, mismo formato de Spring.

Observaciones

- **200 y 201 no son lo mismo.** Abrir cuenta y transferir respondieron 201 (se creó algo nuevo); las consultas respondieron 200.
- **El dinero cuadra.** Ana pasó de 1000 a 800 y Beto de 500 a 700; el total sigue siendo 1500. Nada se creó ni se perdió, solo cambió de cuenta.
- **Precisión de la fecha.** Al crear la transferencia, la fecha traía 9 decimales (...428098444Z); al consultarla después traía 6, redondeados (...428099Z). Al leerla de la base de datos, la precisión es de microsegundos.
- **Idempotencia.** El reintento del SPEI (misma llave) devolvió el mismo id 1, con el monto como 50.00 en vez de 50, y Ana quedó en 750.00, no en 700. También respondió 201, no otro código: la app devuelve el resultado original como si fuera la primera vez.
- **Estado de cuenta.** Muestra DEPOSITO +1000.00 (saldo 1000.00), TRANSFERENCIA_ENVIADA -200.00 (saldo 800.00) y SPEI_ENVIADO -50.00 (saldo 750.00). Cada movimiento guarda su saldo resultante, así que se puede verificar: $1000 - 200 - 50 = 750$. Hay un solo SPEI_ENVIADO, aunque mandé el SPEI dos veces. Los ids son 1, 3 y 5; lo más probable es que los faltantes correspondan a movimientos de Beto, pero no lo comprobé.
- **Avisos.** Hay uno por cada salida de Ana (el SPEI de 50 y la transferencia de 200). El depósito inicial no generó aviso, y el reintento del SPEI tampoco generó uno extra.
- **Formatos de error.** 404, 409 y 422 salieron en formato Problem Details (type, title, status, detail, instance). Los dos 400 salieron con otro formato (timestamp, status, error, path), sin detail, así que no dicen qué campo falta. La explicación está en el Paso 5.

- **400 contra 422.** El 400 fue una petición mal armada (falta un dato). El 422 fue una petición bien armada que rompe una regla de negocio (misma cuenta, saldo insuficiente).
- **Límite de mi comprobación.** Después de los primeros cuatro errores, Ana seguía en 750.00, pero solo revisé la cuenta de Ana. No volví a consultar los saldos después del saldo insuficiente ni del SPEI sin llave.

Paso 4 · Con y sin volumen

Antes de empezar, Ana tenía 750.00. Hice dos pruebas:

A) Sin borrar el volumen

```
docker compose down
docker compose up -d
```

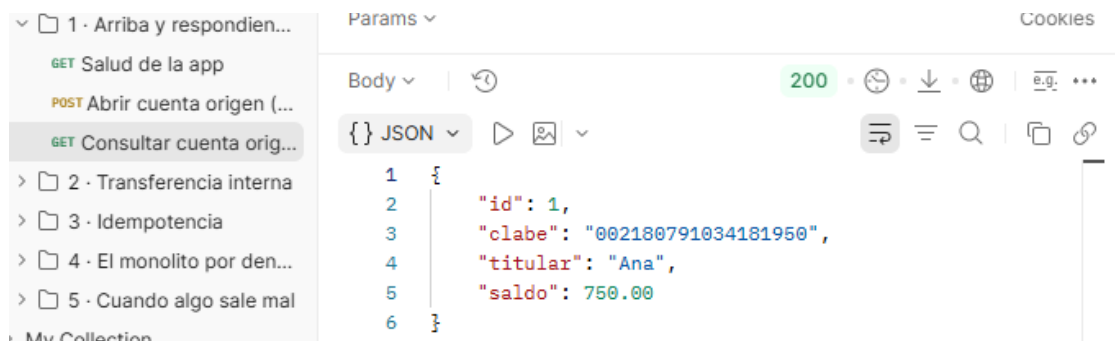
- El down quitó los contenedores y la red, pero el volumen no apareció en la lista de lo que se quitó.
- Después del up, "Consultar origen" respondió 200 con el mismo id, la misma CLABE y saldo 750.00. Los contenedores eran nuevos, pero los datos seguían ahí.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco ((v05.1))
$ docker compose down
[+] down 3/3
✓ Container mexi-banco-app-1 Removed 0.4s
✓ Container mexi-banco-db-1 Removed 0.4s
✓ Network mexibanco Removed 0.2s
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco ((v05.1))
```

docker compose down — sin -v, el volumen no aparece en la lista.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco ((v05.1))
$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app        About a minute ago Up 56
seconds (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s..." db         About a minute ago Up Abou
ut a minute (healthy) 5432/tcp
```

docker compose up -d y ps — ambos healthy otra vez.



Consultar origen — 200, Ana sigue en 750.00.

B) Borrando el volumen

```
docker compose down -v
docker compose up -d
```

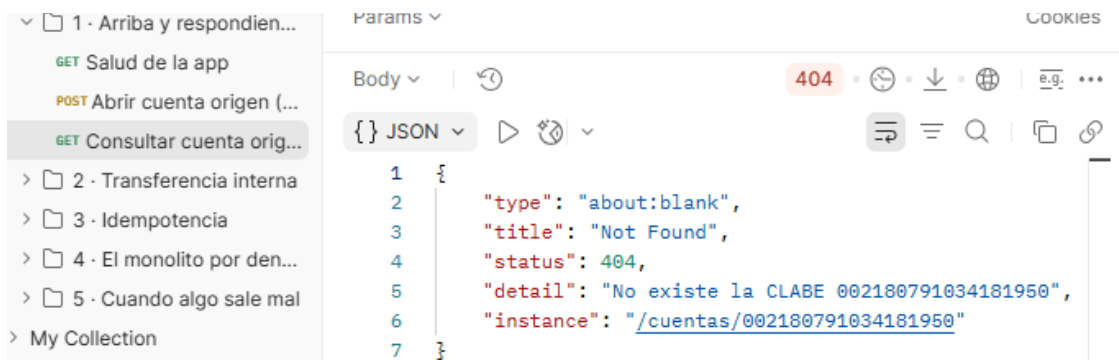
- Esta vez el down sí mostró "Volume mexi-banco_mexibanco_datos Removed", y el up mostró "Volume ... Created": un volumen nuevo y vacío.
- Esperé a que app estuviera healthy (un ps a los pocos segundos todavía decía "health: starting") para que el arranque no afectara mi resultado.
- **"Consultar origen" respondió 404:** "No existe la CLABE 002180791034181950", la misma CLABE de Ana que antes existía.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco ((v05.1))
$ docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1      Removed      0.4s
✓ Container mexi-banco-db-1      Removed      0.3s
✓ Network mexibanco              Removed      0.3s
✓ Volume mexi-banco_mexibanco_datos Removed      0.0s
```

`docker compose down -v` — el volumen sí aparece como *Removed*.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco ((v05.1))
$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app        56 seconds ago Up 49 seco
nds (healthy)       0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s..." db         56 seconds ago Up 55 seco
nds (healthy)       5432/tcp
```

`docker compose up -d y ps` — volumen nuevo, ambos *healthy*.



Consultar origen — 404, la cuenta de Ana ya no existe.

¿Por qué?

El contenedor de la base de datos es desechable: se puede destruir y volver a crear. Lo que conserva el estado es el volumen `mexibanco_datos`, montado en `/var/lib/postgresql/data`. Con `down` se conserva; con `down -v` se borra y los datos se pierden. Es lo mismo que vi en S04 con el volumen `datos_banco`.

Paso 5 · Revisión del código

Abrí el proyecto en VS Code, en `src/main/java/mx/mexibanco`. Tiene los cinco módulos que menciona la guía (cuenta, movimiento, transferencia, notificacion y spei), una carpeta compartido y la clase `MexiBancoApplication.java`. Revisé a fondo el módulo cuenta y la carpeta compartido; de los otros módulos solo vi la lista de carpetas.

Módulo cuenta: sus cuatro capas

Archivo	Capa	Qué vi
CuentaController.java	Controlador	Solo traduce HTTP a llamadas al servicio. Define POST /cuentas y GET /cuentas/{clabe}.
CuentaService.java	Servicio	Aquí están las reglas: abrir, consultar, cargar y abonar.
CuentaRepository.java	Repositorio	Solo vi el nombre del archivo; no lo abrí.
Cuenta.java	Entidad	Solo vi el nombre del archivo; no lo abrí.

Observaciones

- **El 201 sale del controlador.** El método abrir tiene `@ResponseStatus(HttpStatus.CREATED)`; consultar no tiene nada, y por eso responde 200.
- **Dónde nace cada error de negocio.** El 409 sale de abrir (si `existsByClabe` da verdadero), el 404 sale de consultar (si no encuentra la cuenta) y el 422 de saldo insuficiente sale de cargar (si el saldo es menor que el monto).
- **El controlador valida el formato.** El registro `AltaCuenta` marca clabe y titular con `@NotBlank`, y abrir recibe `@Valid`. Por eso el cuerpo sin clabe es un 400: la validación falla antes de llegar a las reglas del servicio.
- **CuentaService es la única clase que toca CuentaRepository.** Según su comentario, transferencia y SPEI mueven saldos llamando a cargar y abonar, y esa frontera es la que se volvería una llamada por red si cuentas se separa en su propio servicio.
- **El saldo es un caché de los movimientos.** Cuando una cuenta nace con dinero, abrir registra un movimiento DEPOSITO con la referencia "saldo inicial"; es el primer movimiento que vi en el estado de cuenta de Ana.

La carpeta compartido y el manejo de errores

- Contiene tres excepciones (`NoEncontradoException`, `ConflictoException` y `ReglaDeNegocioException`) y `ManejadorDeErrores.java`.
- `ManejadorDeErrores` tiene solo tres métodos: `NoEncontrado` → 404, `Conflicto` → 409 y `ReglaDeNegocio` → 422, todos con `ProblemDetail` (RFC 9457). Según su comentario, es el único lugar donde una excepción de negocio se vuelve respuesta HTTP, y por eso los servicios no importan nada de Spring Web.
- **Por qué los 400 salieron distintos.** El manejador no tiene ningún método para errores de validación. Como el 400 del cuerpo sin clabe viene de una validación, no pasa por ahí y sale con el formato por defecto de Spring Boot. Eso último es cómo funciona Spring Boot en general; no lo vi configurado en ningún archivo. No revisé el controlador de SPEI, así que el segundo 400 (SPEI sin `Idempotency-Key`) es consistente con lo mismo, pero no lo comprobé.

Paso 6 · Compose y Dockerfile

docker-compose.yml

- **Servicios:** db (postgres:17-alpine) y app (construida con build: .).
- **Healthcheck de db:** pg_isready cada 5 segundos, con timeout de 3 y hasta 5 reintentos.
- **Healthcheck de app:** un wget a /actuador/health que busca la palabra UP, cada 10 segundos; es el mismo endpoint que probé como "Salud de la app". Esto es consistente con que app se quedara un rato en "health: starting" después de que db ya estaba healthy, como se ve en mis capturas del Paso 4.
- **Orden de arranque:** app tiene depends_on con condition: service_healthy, así que espera a que db esté lista de verdad, no solo creada.
- **Red:** se llama mexibanco (nombre explícito). app encuentra a la base por nombre (DB_HOST: db), sin ninguna IP.
- **Volumen:** el archivo lo declara como mexibanco_datos, pero en la terminal apareció como mexi-banco_mexibanco_datos: Compose le antepone el nombre del proyecto (la carpeta). Es el volumen del Paso 4.
- **Puertos:** solo app publica un puerto (8080:8080); db no publica ninguno, y aun así app le habla por la red interna.
- **Configuración:** las variables van directo en environment, con la contraseña escrita en el archivo. En S04 las saqué a un .env fuera del repo; aquí el proyecto lo hace distinto.

Dockerfile

	Etapas 1 (builder)	Etapas 2 (ejecución)
Imagen base	eclipse-temurin:21-jdk-alpine (JDK, para compilar)	eclipse-temurin:21-jre-alpine (JRE, solo para ejecutar)
Qué hace	Copia pom.xml y mvnw, descarga dependencias, copia src y compila con ./mvnw clean package	Copia solo el .jar de la etapa 1 (COPY – from=builder) y lo arranca

- La imagen final no lleva el código fuente ni Maven, solo el .jar. Es de esperar que pese menos; no medí los tamaños.

- El Dockerfile copia pom.xml antes que src. Así, las dependencias (lo que más tardó en la primera construcción) quedan en una capa que solo se rehace si cambia el pom.xml. Es la explicación probable de por qué la segunda construcción fue tan rápida.
- ENTRYPOINT ["java", "-jar", "app.jar"] coincide con la columna COMMAND de docker compose ps. EXPOSE 8080 solo documenta el puerto; quien lo publica al host es el ports del compose.
- La línea que falló con "mvnw not found" (Paso 1) es RUN ./mvnw dependency:go-offline -B, de la etapa 1.